

Reviewer Pack — Minimal Rank of Quotient Algebras vs Monotone Circuit Lower ...

Entry 46ee38ac1039 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 06:37:01 UTC

Minimal Rank of Quotient Algebras vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: 46ee38ac1039 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 06:37:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quotient Algebras) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For a given k-CLIQUE instance with n vertices, the minimal rank of its quotient algebra by the ideal generated by the monomials corresponding to edges in k-CLIQUE is upper bounded by $O(n^{(1.5-k)})$.’, ‘The statement holds for all instances where $k < 4$ and $n \leq 40$.’, ‘For any instance contradicting this bound, a monotone circuit of size $O(n^k)$ can be constructed that separates the k-CLIQUE language from the empty set.’]

Rationale (proposer’s reasoning):

[‘Quotient algebras provide a way to study the algebraic structure underlying computational problems.’, ‘This conjecture suggests a potential connection between the algebraic complexity of quotient algebras and the complexity of solving k-CLIQUE instances.’, ‘If true, it could lead to new insights into the nature of monotone circuits and their limitations.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f8e72a9553b3f10a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture regarding the minimal rank of quotient algebras for k-CLIQUE instances, results are considered supported if the minimal rank of the quotient algebra is upper bounded by $O(n^{(1.5-k)})$ for all seeds and $n \leq 40$, and falsified if any seed produces a minimal rank greater than $O(n^{(1.5-k)})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quotient algebra" AND "monotone circuit lower bounds"
- "algebraic combinatorics" AND "k-CLIQUE monotone circuits" - "quotient algebra ideal monomials k-CLIQUE" AND "circuit complexity $O(n^k)$ "

Top relevant hits considered: - [s2:2012.03883] Monotone Circuit Lower Bounds from Robust Sunflowers - [s2:2b3cdd1442e58a5bd6096d68f67f129cd64939c0] Automata, languages and programming : 28th International Colloquium, ICALP 2001, Crete, Greece, July 8-12, 2001 : procee

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_clique(n, k):
    if n - k < k * (k - 1) // 2:
        raise ValueError("Sample larger than population or is negative")
    remaining_edges = random.sample([(i, j) for i in range(k, n) for j in range(i + 1, n)], n - k)
    clique_edges = [(i, j) for i in range(k) for j in range(i + 1, k)]
    return clique_edges + remaining_edges

def compute_quotient_algebra_rank(edges):
    n = len(set(u for u, v in edges) | set(v for u, v in edges))
    algebra = [[0] * n for _ in range(n)]
    for u, v in edges:
        algebra[u][v] = 1
        algebra[v][u] = 1
    rank = 0
    for i in range(n):
        if any(algebra[i][j] != 0 for j in range(i)):
            rank += 1
            for j in range(n):
                if algebra[j][i] != 0:
                    for k in range(n):
                        algebra[j][k] -= algebra[i][k]
    return rank
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        for _ in range(5):
            try:
                k = random.randint(2, min(n - 1, 3))
                edges = generate_k_clique(n, k)
                rank = compute_quotient_algebra_rank(edges)
                if rank > 0_n_1_5_minus_k(n, k):
                    return {
                        "metric_name": "Quotient Algebra Rank",
                        "metric_value": rank,
                        "instances_tested": 1,
                        "conjecture_holds": False,
                        "counterexample": f"n={n}, k={k}, rank={rank} > 0(n^(1.5-k))"
                    }
                results.append(rank)
            except ValueError as e:
                return {
                    "metric_name": "Quotient Algebra Rank",
                    "metric_value": None,
                    "instances_tested": 0,
                    "conjecture_holds": False,
                    "counterexample": str(e)
                }
    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = Fraction(len([r for r in results if r <= 0_n_1_5_minus_k(n, k)]), len(results))
    return {
        "metric_name": "Quotient Algebra Rank",
        "metric_value": mean,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction == 1,
        "counterexample": ""
    }

def 0_n_1_5_minus_k(n, k):
    return n ** (1.5 - k)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + [101, 103, 105]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = Fraction(len([r for r in results if r["conjecture_holds"]]), len(results))

    if support_fraction == 1:

```

```

    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif support_fraction >= Fraction(4, 5):
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture"])
    print(f"RESULT: FALSIFIED counterexample=\"{{result['counterexample']}}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

=5, k=2, rank=3 > 0(n^(1.5-k))"
TRIAL: {"seed": 503, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 547, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 593, "metric_name": "Quotient Algebra Rank", "metric_value": None, "instances_tested": 1
k))"}
TRIAL: {"seed": 631, "metric_name": "Quotient Algebra Rank", "metric_value": None, "instances_tested": 1
k))"}
TRIAL: {"seed": 677, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 727, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 773, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 821, "metric_name": "Quotient Algebra Rank", "metric_value": None, "instances_tested": 1
k))"}
TRIAL: {"seed": 877, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
TRIAL: {"seed": 929, "metric_name": "Quotient Algebra Rank", "metric_value": 3, "instances_tested": 1
k))"}
RESULT: FALSIFIED counterexample="n=5, k=2, rank=3 > 0(n^(1.5-k))" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test has only considered a very small range of n (up to 40) and k (less than 4), which may not be representative of the conjecture's validity for larger values of n . Additionally, the metric value is reported as None for some trials, indicating potential issues with the metric definition or computation.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: A counterexample was found where the conjecture does not hold: for $n=5$ and $k=2$, the minimal rank of the quotient algebra is 3, which exceeds the bound | next: Further investigation is needed to determine if the conjecture holds for larger values of n and k , or if there are specific conditions under which it fails.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16324
2	propose	ollama_remote	glm4:latest	0	0	14094
3	preregistration	ollama_remote	glm4:latest	0	0	9708
4	novelty	ollama_remote	glm4:latest	0	0	11256
5	novelty	ollama_remote	glm4:latest	0	0	9547
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16733
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12171
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42135
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15193
10	critic	ollama_remote	glm4:latest	0	0	50958
11	judge	ollama_remote	glm4:latest	0	0	11078

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 209196 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/46ee38ac1039.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46ee38ac1039.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/46ee38ac1039.tar.gz` (if generated)